Motion Planning of a Robot in Real-Time based on the General Model of Humanoid Robots

Aleksandar Batinica, Mirko Raković, Miroslav Zarić, Branislav Borovac, Milutin Nikolić
University of Novi Sad
Trg Dositeja Obradovića 6
Novi Sad, Serbia
Email: [batinica, rakovicm, miroslavzaric, borovac, milutinn]@uns.ac.rs

***Abstract*—Humanoid robots are the most illustrative representatives of complex cyber-physical systems. They are high-dimensional and highly coupled non-linear systems. Moreover, walking humanoid robots are underactuated with friction-limited unilateral contacts with the ground. These facts justify the need for hard real-time control of a robot's motion. Even the smallest delay in calculating the reference motion or setting the output control variable can lead to a catastrophic outcome. For this reason, most of the existing robotic control systems are real-time and custom made, intended for a specific robot (or family of robots). There is no platform offering a general tool to design the model of versatile configurations of robots and to use it in the model-based control algorithms in hard real-time.**

In this paper, a software tool for modelling of an articulated system consisting of several open branched kinematic chains for a hard real-time OS is proposed. The model of the robot can possess an arbitrary number of links. The software is developed on top of the QNX Neutrino real-time operating system, thus providing the precise execution of time-critical events. In the experiment, the advantages of using the general model by modelling one real robotic configuration with multiple branched kinematic chains will be shown. The processing performance during the on-line motion planning will be tested on a real-time platform running on ARM Cortex-A8 processor. The results will be compared with similar software running in a non-real-time simulation environment.

I. Introduction

The embedded computer hardware and software used to control modern humanoid robots or to analyse the biomechanics of human movements is becoming increasingly complex with the constant requirement to increase the efficiency of computation and to reach the constraints imposed by the operation in a real-time environment [1]. Nowadays, the robot control systems are distributed, mainly embedded real-time and data intensive systems. With the increasing complexity of robotic systems, modularity is considered an important factor. As a result of these requirements, the robotic control software and hardware changed its structure from a single large scale application running on a single processor to distributed architectures [2].

The control system of humanoid robots typically consists of numerous interconnected computers and microcontrollers operating at multiple levels, from the low-level control of actuators to the high-level cognitive and motion planning systems. Depending on the exact location and role of any given control system segment, different requirements with respect to the execution speed of control loops, the exact timing of event occurrence, operating system requirements (including the requirement to use non-OS firmware) etc, can be obtained. This type of control system architecture exploits the advantages of parallelism where various control tasks are executed in parallel in order to perform the necessary computations in real-time and with the imposed reliability constraints. On the contrary, this approach introduces the complexity in the integration of all the segments into a functional and reliable system as a whole.

To cope with the increasing complexity, in the last decade the robotic control software was usually designed as a multilayer architecture with different responsibilities allocated to each level in the structure. The usual architecture has at least three layers. The lowest level is the closest to the hardware. It is responsible for interfacing with the sensors and also acts like low-level motion control of the actuators driving the joints. Software in this layer is generally developed without using an operating system because the response time has to be as short as possible and there is no need for distributed tasks. The cycle time of the control loop on this level is generally not greater than 1ms.

The next layer keeps the information of the structure of the robotic system and is responsible for motion planning based on the input from the sensors and the kinematic and dynamic model of the robot. The software is performing several tasks on this level : motion planning, higher level motion control of the arms, legs and other parts of the robot's body, communication with the low-level control systems etc. Since there are distributed tasks on this level, and some of them are required to be executed within a timely precise control loop, it is preferable to develop software at this level on top of a real-time operating system. There are different opinions on how small the cycle time of the control loop on this level should be. In [3] the authors stated that for operating in a dynamic and uncertain environment, the motion planning must be generated online. If an autonomous system should quickly react to the sudden change in the situational awareness, the planning interval must be shorter than 100 milliseconds.

Finally, the highest layer in the architecture is used for the implementation of the highest level of the control systems,

978-1-5090-2866-5/16/$31.00 ©2016 IEEE

such as decision making, reasoning and implementation of cognitive features of the robot. Based on the type and the complexity of the tasks allocated to the third layer it is not required to use a real-time OS, and conventional operating systems are usually the best choice. Also, the requirements for the time intervals of the control loop are less demanding.

Some examples of a layered architecture for controlling a robot's motion are presented in [4], [5], where the authors used the Emotionally GrOunded architecture to develop a module-based behaviour selection architecture for a personal robot intended for a real-world environment. In [6] the authors developed a customised robot framework that allows the realization of distributed robot software components such as communication properties, initialization and error handling, mechanisms for state implementations, interface definitions and customizable building blocks for high-level robot control. In its white papers [7], the company National Instruments proposed a software architecture, for mobile robots, that takes the form of a three- to a four-layer system, i.e. driver layer, a platform layer, algorithm layer, and user interface layer. The layers are implemented in such a way that each layer in the software depends only on the specific system, hardware platform, or end goal of the robot and remains completely blind to the contents of the layer above or below it.

In [8] the authors presented the Coupled Layered Architecture for Robotic Autonomy (CLARAty). The intention is to develop a framework for generic and reusable robotic components that can be adapted to a number of heterogeneous robot platforms. CLARAty consists of two distinct layers: the Functional Layer and the Decision Layer. The Functional Layer defines the various abstractions of the system and adapts the abstract components to real or simulated devices. The Decision Layer provides the system's high-level autonomy, which reasons about global resources and task constraints. Later, in [9] the authors addressed the difficulties of developing reusable robotic software because there is no universal agreement upon a definition of what a robot is. Over the past half-century, robots took many forms. Some were inspired by their biological counterparts such as humanoids, dogs, snakes, and spiders. Others were designed for particular domains such as manufacturing, medical, service, military, or space applications. They took the form of arms, wheeled robots, legged robots, hoppers, blimps, underwater vehicles, subsurface diggers, and even reconfigurable robots.

In the seminal paper-on the so called robotic operating system (ROS) [10], [11] it is nicely pointed out that many researchers have previously created a wide variety of frameworks to manage complexity and facilitate rapid prototyping of software for experiments, resulting in the many robotic software systems currently used in the academia and industry. Each of these frameworks was designed for a particular purpose. The intention behind the ROS framework is to offer a software that puts together as many components as possible, used for the development of the robotic control system. Currently, the ROS composes of several core components which are grouped in the communication infrastructure, robot-specific features and tools packages. Particularly interesting for this paper is the robot geometry library which is used to manage coordinate transform data for robots with more than one hundred degrees of freedom and update rates of hundreds of Hertz. The ROS is mainly suitable for mobile robots which can be seen from the list of robots using this framework. Only several humanoid robots (NAO, PR2, ROMEO, REEM and TUlip) are using the ROS as a platform to develop its control system. One of the possible reasons is that ROS is initially not designed to work on a hard real-time operating system, and for reliable, precise and safe motion of humanoids this is a critical requirement.

Some research groups where dealing with the development of software design patterns to overcome the problem of rewriting the code for each robotic platform from the beginning [12], [13]. The main identified problem was that the available software components were not used because they were designed for specific platforms, operating systems or communication protocols and conventions.

In this paper, the need for a software framework that will be a general modelling tool for humanoid robots tailored for execution on a hard real-time operating system is addressed. This will offer a possibility to define and use the model of all types of robots which can be represented in a form of branched kinematic chains consisting of rigid bodies i.e. links, connected with joints that have one degree of freedom. In the perspective of layered architectures of the control system, the software framework proposed in this work belongs to the second layer. In addition to the motion planning and model calculation, the software on this level should be the bridge between the high-level cognitive reasoning and decision making, and the low-level control system, such as [14], for controlling the joint actuators and for sensing.

The paper is organized as follows. First, in Section II the concept of the general model with the background mathematical tools is described. Afterwards, in Section III the description is given of the classes developed during the implementation of the general model for running on the QNX Neutrino real-time operating system. The validation tests of the implemented model are performed by comparing the results obtained from a simulation environment. The test is described and the results are shown in Section IV. In the end, in Section V, the paper is concluded with a short review of the achieved results and the possibilities for future work are discussed.

II. General model of humanoid robots

The general model of humanoid robots used in this paper is described in [15]. The model comprises of links which are connected with joints and thus form kinematic chains. Between two successive links, only a joint with one degree of freedom (DOF) can be introduced. In order to model a joint with N DOFs, N links are required. When modelling multiple DOF joints, the length of the links between the first and the last should be set to zero. This way the axes of rotation of the links interconnect in one point forming a multiple DOF joint. To

mathematically describe the links, which are the basic building blocks of the general model, several vectors are introduced (1). Each link has one input axis of rotation $\vec{e_i}$ which is the rotational DOF of that link. The links also have one or several output axes of rotation $\vec{e^n}_{ij}$ where n denotes the ordinal of the output axis. Both the input and output axes of rotation are unit vectors. Kinematic chain branching is enabled by having multiple output axes. Beside the axes of rotation associated with the link, each link has its own coordinate frame placed in the centre of mass (COM). The input axis of rotation and COM are connected via a vector that is named the direct vector of a link $\vec{r^d}_i$ where i is the index of the link. Similarly, the output axis (or axes) of rotation are connected to the COM via vectors that are named reverse vectors of a link $\vec{r^r}_{ij}$ where i is the index of the link and j is the ordinal of the outgoing vector. Both the direct and reverse vectors magnitude should describe the real distance between the input and output axes and the COM. All the vectors describing a link are referenced to its own coordinate frame. Along with the kinematic parameters of the link, the parameters that will be used to build the dynamic model of the robot are associated with each link. The dynamic parameters of the link are the mass and 3×3 inertia tensor.

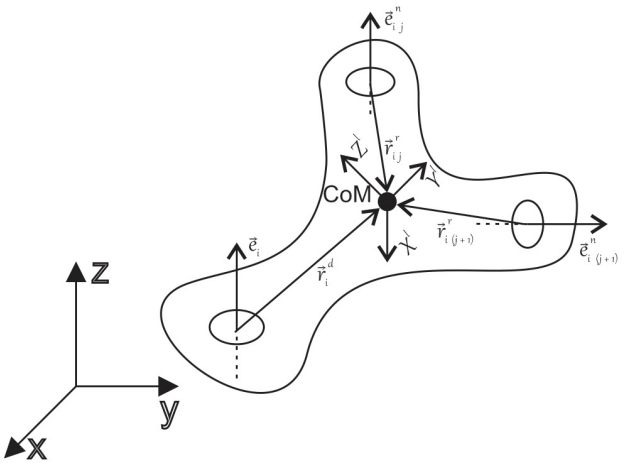


Fig. 1: The vectors used for the mathematical descripton of a link.

In general, each link can be positioned anywhere in space by placing its own coordinate frame at a desired position in the global coordinate frame. However, this is not true if the links are connected. A link is considered connected if its input axis of rotation is connected to another links output axis of rotation. In that case, the position and orientation of the link depend on the position and orientation of the previous link thus forming a kinematic chain. The angle in the joints between the links will define the position and orientation of each link in the kinematic chain. The only link that is not connected is the base link. The base link is the robot's root link from which all the kinematic chains start. Unlike other links, whose position depends on the value of angles between the links, the location of the base link is determined with six introduced coordinates which define its position and orientation referenced to the global coordinate frame.

A. Forming the model

The forming of the model is done by assuming all the angles of the joints are zero, i.e. in the robot's zero pose. In this pose the projections of $\vec{r^d}_i$ and $\vec{r^r}_{i-1,j}$ on the surface that is normal to the vector $\vec{e_i}$ should be collinear. As an outcome of this condition, the transformation matrices between the links coordinate frames are identity matrices when the robot is in a zero pose.

When the values of the joints are non-zero, the robot is moving. During the robot's motion, the transformation matrices between two coordinate frames associated with two consecutive links are changing and are depending on the value of the angle between the links. To calculate the transformation matrices between the coordinate frames of two consecutive links for an arbitrary angle value, the Rodrigues' rotational formula is used:

$$\vec{V}_{ik} = \vec{V}^0_{ik} cosq_i + (1 - cosq_i)(\vec{e^n}_{i-1,j} \cdot \vec{V}^0_{ik})\vec{e^n}_{i-1,j} + (\vec{e^n}_{i-1,j} \times \vec{V}^0_{ik}) sinq_i, k = 1, 2, 3;$$

Where i is the link index, $\vec{V}_{ik}$ is the kth column on the transformation matrix, $\vec{V}^0_{ik}$ is the kth column of the transformation matrix in the zero pose. The angle in the joint is denoted with q_i and $\vec{e^n}_{i-1,j}$ is the $i-1$th links jth output axis of rotation.

Now that all the transformation matrices between the coordinate frames of consecutive links are known, the transformation between any two coordinate frames of the links in the same kinematic chain can be calculated. For this calculation, the kinematic chain is used as a layer of abstraction in which the sequence of links is defined. Additionally, by multiplying transformation matrices in the same order they are defined in the kinematic chain, the transformation matrix between the global coordinate frame and the coordinate frame associated with each link can be calculated. This is required when it is needed to express the vectors that describe the links in the global coordinate frame. The steps described so far are enough to solve the direct kinematic problem, where based on the generalized coordinates in the joint space (i.e. values of the angles), the position of the end-effector link in the task space can be calculated.

B. Calculating the Jacobian and the solution of the inverse kinematic problem

The output from the motion planning algorithms, which are commonly based on the input from a vision system, are providing the coordinates to the robot in the task space. Thus, the problem of calculating the motion of the joints, based on the desired trajectories given in the task space has to be solved. This is known as the inverse kinematic problem. The solution of the inverse kinematic problem is used in many robotic applications [16], [17]. The inverse kinematic solvers are also used for computer graphics [18], wearable devices [19] and other similar areas.

To utilize the numerical solution of the inverse kinematics or to calculate the motion of the joints based on the desired task space trajectory the Jacobian matrix should be calculated for the end-effector link. The Jacobian matrix is composed of first-order partial derivatives of the position of the end-effector with respect to all the joints' coordinates, and thus it represents the relation between the joints' velocities and the velocity of the end-effector in the task space. The size of the Jacobian is $6 \times (N + 6)$, where N is the number of DOFs in the kinematic chain and the additional six is a consequence of the base link being defined with six additional coordinates. The Jacobian can be calculated for any two coordinate frames, but most of the time it is required to be calculated between the global coordinate frame, and the coordinate frame of the end-effector. It can be represented as:

$$J = \begin{bmatrix} \vec{\beta}_{i1} & \vec{\beta}_{i2} & ... & \vec{\beta}_{ii} & \vec{0} & ... & \vec{0} \\ \vec{\alpha}_{i1} & \vec{\alpha}_{i2} & ... & \vec{\alpha}_{ii} & \vec{0} & ... & \vec{0} \end{bmatrix} \quad (1)$$

where i is the ordinal of the link inside the kinematic chain for which the Jacobian matrix is being calculated. Most of the time the Jacobian matrix is calculated for the last link in the chain. If during the motion planning, some of the joints should not be used for a given task, the column vector in the Jacobean matrix associated with those joints should be set to zero. The $\vec{\alpha}$ and $\vec{\beta}$ vectors describe the effect of the linear and angular velocities of the previous links in the chain on the current link's angular and linear velocity, respectively. They can be calculated the following way:

$$\vec{\alpha}_{ik} = \vec{\alpha}_{(i-1)k}, k = 1, ..., i-1, \quad (2)$$

$$\vec{\alpha}_{ii} = \vec{e}_k, \quad (3)$$

$$\vec{\beta}_{ik} = \vec{\beta}_{(i-1)k} - \vec{\alpha}_{(i-1)k} \times \vec{r}_{i-1} + \vec{\alpha}_{ik} \times \vec{r^d}_i, k = 1, ..., i-1, \quad (4)$$

$$\vec{\beta}_{ii} = \vec{\alpha}_{ii} \times \vec{r^d}_i, \quad (5)$$

where i is the ordinal of the current link which is analysed, and k is a counter that goes through all the previous links in the kinematic chain.

For given angular velocities in the joints, with the use of the Jacobian matrix, the velocities of the end effector can be calculated. However, from the motion planning algorithm, the desired velocities of the end-effector are given, and the angular velocities in the joints should be calculated. For solving this, an inverse of the Jacobian matrix is needed. The inverse of a matrix only exists for a square matrix. Since the dimensions of the Jacobian are dependent on the configuration of a robot, it is a common case that the matrix is not of square shape. In the case of humanoid robot's arms and legs, the number of DOFs is usually more than six, giving a rectangular Jacobian matrix. In such a case the most often used approaches are based on the transposed Jacobian matrix and the pseudo-inverse Jacobian.

III. Implementation of the General Model on a Real-Time Operating System

As it was mentioned in the introduction, some tasks in controlling a robot are best suited to be run on a real-time operating system. The motion planning based on the implementation of the general model of a robot is such a task because it is time critical. Also, on this level, other tasks are needed to be done in parallel, such as the communication with a higher level supervisory controller and a lower level motor driver.

A. Description of the hardware/software platform

The processor that the RTOS is running on is Texas Instruments' AM3358 Sitara. This processor is based on the ARM Cortex-A8 processor. The processor can run up to 1GHz, it has 32KB of L1 Cache and 256KB of L2 Cache. The experiments in this paper are conducted at 720MHz. Alongside the processor, there is 512MB DDR3L RAM memory clocked at 800MHz.

The RTOS used is the QNX Neutrino. QNX Neutrino is a robust Unix-like operating system. Its robustness is a result of the drivers, protocol stacks, file systems and applications running in a memory-protected user space which is outside the kernel. If any component fails it can be restarted without affecting the kernel or other processes. Its main use is for embedded systems but is often used in the automotive industry where the high reliability and robustness is a key property. QNX Neutrino supports the majority of conventional processor architectures such as x86, MIPS, powerPC, ARM, etc. The reason why it was chosen for this task is because of its high reliability and robustness and the support for many processor architectures which is good in case it is needed to move to a higher-end processor.

The software development was done in QNX's IDE, QNX Momentics. QNX Momentics is an Eclipse-based IDE which has some vendor specific options integrated into it. It supports C and C++, and it has a debugger, system builder, application profiler, system profiler and memory analysis options. The compiler it uses is an optimised version of GCC that they have named QCC. The compiler supports the C++11 standard.

The programming language used is C++. The only non-standard library that was used is the free version of AlgLib. AlgLib is a linear algebra library, which consists of a few classes such as ones for vectors and matrices, and procedures for working with vectors and matrices. However, to maintain and optimise the efficiency of the code, new procedures have been created since AlgLib is designed to work with vectors and matrices of all sizes, which introduces an undesirable overhead. In the general model implementation, vectors predominantly have the size of 3×1, while most of the matrices are 3×3, with some exception such as the Jacobian matrix, desired end effector velocity etc. For the calculation of the pseudo-inverse matrix, a new procedure is implemented that uses the Lower-Upper decomposition implementation in AlgLib.

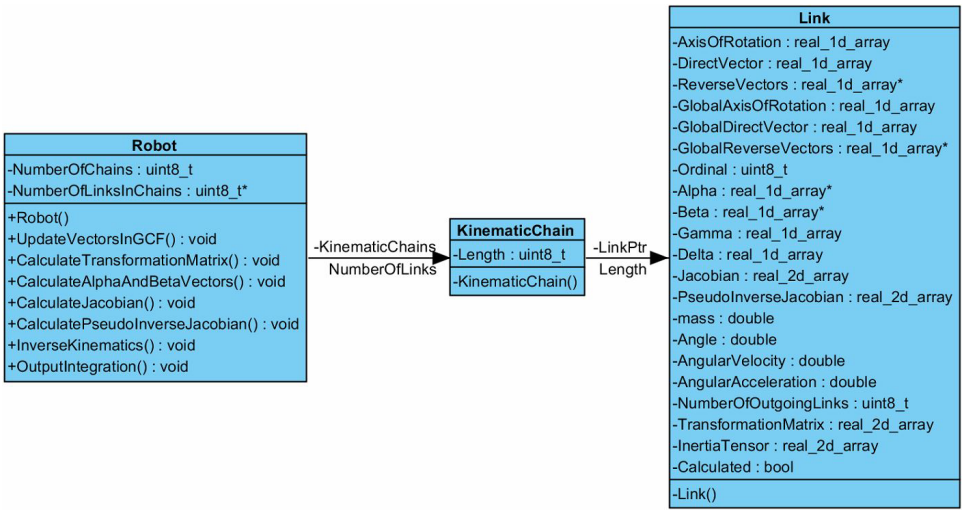


Fig. 2: The UML class diagram of the implemented classes of the general model.

B. Implementation of classes and their use in the main control loop

As mentioned before the programming language used is C++. C++ is an object-oriented programming (OOP) language, but it is not a pure OOP language. The classes created to implement the general model are `Link`, `KinematicChain` and `Robot`. The UML class diagram is shown in Fig. 2. All the attributes of the classes are private. The user interface is implemented through the `Robot` class. Each class has its own interface, but after profiling the application it was clear that the communication between classes through their interfaces takes a noticeable amount of time. Thus, the classes were given the friend access to each other so they could communicate faster between themselves.

In the class diagram of the class `Link` (see Fig. 2) all the attributes of a link described in Section II can be found. The vectors are objects of the `real_1d_array` class, while the matrices are objects of the textttreal_2d_array class. Both of these classes are part of the AlgLib library. Some attributes depend on other attributes so they are declared as pointers, so later on a required amount of memory can be allocated to them. Examples of this are the `ReverseVector` and `GlobalReverseVector` which depend on the `NumberOfOutgoingLinks`, and `Alpha` and `Beta` which are arrays of vectors, where the length of the arrays depends on the length of the corresponding kinematic chain. The `Calculated` attribute serves as an indicator of whether or not the link's attributes that need calculations to be done, are already done. The need for this will be explained in the next paragraph.

The `KinematicChain` class has two attributes. One is the length of the chain, which is a parameter used for accessing the links that belong to the kinematic chain, and the second is a pointer to a dynamically allocated array of pointers to a `Link` object. The length of the array depends on the length of the kinematic chain. The reason for using pointers to objects is the fact that some links may be found in more than one kinematic chain. Therefore, undesirable object cloning and possible negative side effects (manipulation on a local copy rather than actual link object) are avoided by using pointers to already existing Link objects representing the specific link. Multiple pointers in different kinematic chains can point to the same Link, perfectly reflecting actual physical dependency. This approach, even when it would not cause any other side effect, helps to save memory, which can be of critical importance in embedded systems, and also reduces the number of calculations needed. For example, if one link is found in four kinematic chains it is enough to do the calculations once for the link with the address the pointer is pointing to. The next time the program reaches a pointer that points to the same memory (representing the same link in different kinematic chains) the calculations are already done. However, the program does not know that so a flag `Calculated` is declared as an attribute of the class `Link`. This flag is set the first time the calculations are done in a single iteration of the control loop, and reset at the end of the iteration.

The `Robot` class has three attributes, of which two are used to hold information about the total number of distinct kinematic chains and links in the model. The third one is a pointer to an object of the class `KinematicChain`, which is a dynamically allocated array of kinematic chains. The methods of the `Robot` class may be logically grouped into two groups. The first group contains the methods used to set the model properties and to allocate necessary resources (memory). These methods are used to define the model. The second group consists of the methods that are used for the calculation the kinematic and dynamic properties of the model. The main difference is that the methods from the first group are called only at the beginning of the program to construct the initial model, while the methods from the second group are called in each iteration of the control loop. The basic workflow of the program is shown in Fig. 3.

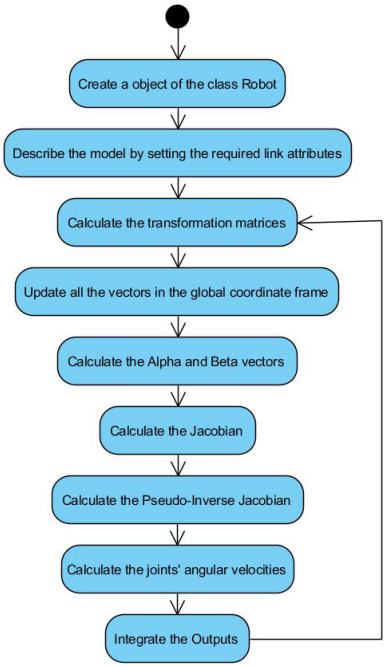


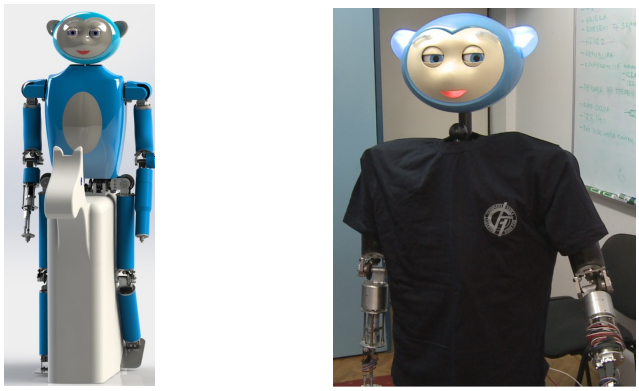
Fig. 3: The workflow of the program with model initialization and the main control loop.

IV. Validation of the Implementation of the General Model

The validation of the implementation of the general model will be based on the execution of the motion planned in the task space. The output data will be compared with the results obtained from the mature simulation environment in which the model of the robot can be built with the similar general model concept originally introduced in [15]. The simulation environment is being used and validated in many research experiments such as biped locomotion in unstructured environment [20] and whole body motion control with task prioritization framework [21]. The robot model used in this paper represents the implementation of the upper body with two arms of the real robot MARKO, and it will be described in the following subsection.

A. Humanoid Robot MARKO

At the University of Novi Sad, the robot MARKO is under development. Marko is an assistive robot intended to be involved in therapeutic treatments of children with cerebral palsy. The robot will be placed on a mobile platform that enables movement in its environment. The purpose of the legs and the arms is for demonstrating the gross motor skill exercises while the arms will also be used for demonstration of fine motor skill exercises. In order to communicate verbally and non-verbally, the robot has an integrated cognitive dialogue system, which integrates subsystems for speech recognition and synthesis, the ability to express emotions, gestures and body language whose purpose is to seek, establish and maintain social interaction.



(a) (b)

Fig. 4: The humanoid robot MARKO a) CAD model b) real robot.

The arms of the robot MARKO have 7 DOFs each, closely approximating a human arm. Joint ranges, links' dimensions, as well as their masses and position of the centres of mass, are designed to match the average human arm parameters to the maximum extent possible [22]. In Fig. 4a a CAD model of the robot's arms are shown and in Fig. 4b the current version of the robot is shown.

B. The model of the Robot MARKO and Motion Planning in the Task Space

The kinematic structure of the robot MARKO's upper body is shown in Fig. 5. The structure is composed of three kinematic chains representing the left arm, the right arm and the neck with the head. The torso to which the arms are attached has been adopted as the base link.

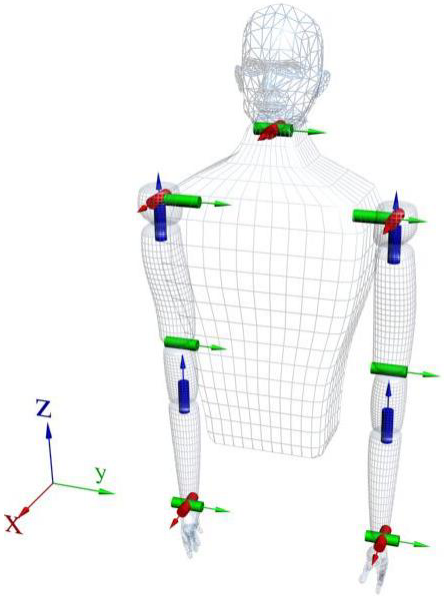


Fig. 5: The kinematic structure of the arms of the robot MARKO.

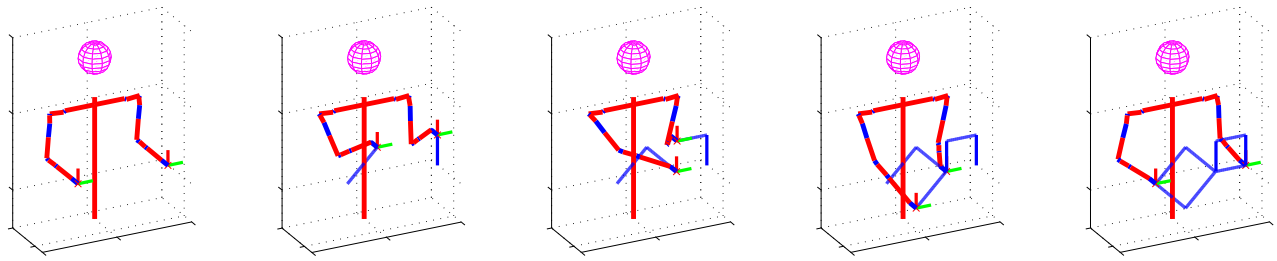


Fig. 6: The poses of the robot every 1000 iterations when executing square paths with the end-effector.

The neck consists of 2 DOFs and each arm consists of a shoulder joint with 3 DOFs (approximated with three rotational joints whose axis of rotation intersect at one point), elbow joint with 1 DOF and a wrist joint with 3 DOFs (whose axis of rotation also intersect at one point). The state vector **q** consists of the 6 coordinates that represent the position and orientation of the base link, the 7-elements vector consisting of relative angles between two adjacent links in both left and right arm and 2-element vector representing the angles in the neck. So, the total number of DOFs, of robot MARKO, is 22.

To validate the implementation of the general model on a hard real-time OS, a verification test is performed in the simulation environment and on an RTOS using the implemented general model. The initial pose of the robot in both cases is the same and it is shown on the top left stick diagram in Fig. 6. The validation test task given to the robot is to move its end-effector to execute linear paths. The velocity of the end-effectors associated to the left and right arm changes in the following way. During the first 1000 iterations (dt=1ms) the velocity of the right end-effector is set to $[0, 0.2, 0.2, 0, 0, 0]^T$ and of the left to $[0, 0, 0.2, 0, 0, 0]^T$. Then for the next 1000 iterations, the velocities of the right and left end-effector are set to $[0, 0.2, -0.2, 0, 0, 0]^T$ and $[0, -0.2, 0, 0, 0, 0]^T$ respectively, for next 1000 iterations the velocities of the right and the left end-effector are set to $[0, -0.2, -0.2, 0, 0, 0]^T$ and $[0, 0, -0.2, 0, 0, 0]^T$, and finally for the last 1000 iterations the velocities of the right and left end-effector are set to $[0, -0.2, 0.2, 0, 0, 0]^T$ and $[0, 0.2, 0, 0, 0, 0]^T$. The unit of linear velocities is [m/s] and for the angular velocities is [rad/s]. The resulting trajectory of the end-effector with the poses of the robot at the moment when velocities of end-effectors changes are shown in Fig. 6.

C. Comparison with the Results from Simulation Environment

The results of the motion planning calculated from the general model implemented in C++ on an RTOS were compared with the results obtained from a MATLAB implementation of the model. The results from MATLAB were used as a reference since many published simulation experiments over the years were based on the results that came out from its use.

The task that the robot should execute is to move both hands simultaneously, following a square outline. The square outlines are coplanar and are rotated by $45°$ one from another (see Fig. 6). The results from the MATLAB simulation and from the application running on RTOS was recorded after every iteration and the difference between them is calculated. The errors between two applications are shown in Table I, in which q denotes the angle in the joint and $\dot{q}$ denotes the angular velocity of the joint. The results given are the maximum of maximum errors of all the robots joints.

TABLE I: Maximum values of errors

		Max. Abs. Err.	Max. Rel. Err.
Left Arm	q	$1.44 \cdot 10^{-15}[rad]$	$2.63 \cdot 10^{-9}$
	$\dot{q}$	$7.1 \cdot 10^{-15}[\frac{rad}{ms}]$	$2.25 \cdot 10^{-11}$
Right Arm	q	$1.44 \cdot 10^{-15}[rad]$	$2.672 \cdot 10^{-10}$
	$\dot{q}$	$3.99 \cdot 10^{-14}[\frac{rad}{ms}]$	$7.4 \cdot 10^{-12}$

The test was run for 4000 samples. The average time needed to calculate 4000 samples on RTOS was $17.324s$. That means that one iteration for a 22 DOF robot took $4.3ms$ on average. The maximum absolute variation of the time was $90\mu s$ per iteration.

V. Conclusion

In this paper, the implementation of the general model for modelling the humanoid robots suitable for use in real-time operating systems is shown. The validation test with the motion planning in the task space shows that the output results do not deviate significantly from the results obtained when using the simulation in MATLAB.

The implemented code was made free from overhead by avoiding unnecessary loops, redundant tests, branching, and function calls. The average time needed to execute one cycle for 22 DOF robot with three kinematic chains is 4.3ms. This brings us to the conclusion that it is a very much acceptable cycle time on the level of motion planning and calculating the reference joint trajectories. It should be noted here that there are much more powerful embedded processors on the market than the one used for the experiment in this paper.

The future work will be directed towards the further development of the general model to cover the calculation of the robot's dynamical model, the model of a contact with the objects from the environment and the communication with the other segments of the control system. Advanced motion planning algorithms will be implemented and will be, in the first place, tested for the robot MARKO. Also, the attention

will be given to the proper selection of hardware platform as well as to further optimisation of the code to achieve more efficient calculations of the models.

ACKNOWLEDGMENT

This work was supported in part by the Ministry of Education, Science and Technological Development of the Republic of Serbia under Grant III44008 and in part by the Provincial Secretariat for Science and Technological Development of AP Vojvodina under Grant 114-451-660/2015-02. The authors would also like to thank the company QNX for providing the licenses for the QNX Neutrino RTOS.

REFERENCES

[1] van den Bogert, Antonie J., Thomas Geijtenbeek, Oshri Even-Zohar, Frans Steenbrink, and Elizabeth C. Hardin. "A real-time system for biomechanical analysis of human movement and muscle function." Medical and biological engineering and computing 51, no. 10 (2013): 1069-1077.

[2] Brugali, Davide, and Azamat Shakhimardanov. "Component-based robotic engineering (part ii)." Robotics and Automation Magazine, IEEE 17, no. 1 (2010): 100-112.

[3] Kuwata, Yoshiaki, Justin Teo, Sertac Karaman, Gaston Fiore, Emilio Frazzoli, and Jonathan P. How. "Motion planning in complex environments using closed-loop prediction." In Proc. AIAA Guidance, Navigation, and Control Conf. and Exhibit. 2008.

[4] Hoshino, Yukiko, Tsuyoshi Takagi, Ugo Di Profio, and Masahiro Fujita. "Behavior description and control using behavior module for personal robot." In Robotics and Automation, 2004. Proceedings. ICRA'04. 2004 IEEE International Conference on, vol. 4, pp. 4165-4171. IEEE, 2004.

[5] Aoyama, Kazumi, and Hideki Shimomura. "Real world speech interaction with a humanoid robot on a layered robot behavior control architecture." In Robotics and Automation, 2005. ICRA 2005. Proceedings of the 2005 IEEE International Conference on, pp. 3814-3819. IEEE, 2005.

[6] Vahrenkamp, Nikolaus, Mirko Wächter, Manfred Kröhnert, Peter Kaiser, Kai Welke, and Tamim Asfour. "High-Level Robot Control with ArmarX." Proceedings of the Informatik (2014): 1283-1294.

[7] National Intruments White papers: "A Layered Approach to Designing Robot Software", Publish Date: Apr 10, 2012, http://www.ni.com/white-paper/13929/en/, access: Oct 21, 2015.

[8] Nesnas, Issa AD, et al. "CLARAty and challenges of developing interoperable robotic software." Intelligent Robots and Systems, 2003.(IROS 2003). Proceedings. 2003 IEEE/RSJ International Conference on. Vol. 3. IEEE, 2003.

[9] Nesnas, Issa AD. "The claraty project: coping with hardware and software heterogeneity." In Software Engineering for Experimental Robotics, pp. 31-70. Springer Berlin Heidelberg, 2007.

[10] Quigley, Morgan, Ken Conley, Brian Gerkey, Josh Faust, Tully Foote, Jeremy Leibs, Rob Wheeler, and Andrew Y. Ng. "ROS: an open-source Robot Operating System." In ICRA workshop on open source software, vol. 3, no. 3.2, p. 5. 2009.

[11] Garber, Lee. "Robot OS: A new day for robot design." Computer 12 (2013): 16-20.

[12] Côté, Carle, Dominic Létourneau, François Michaud, and Yannick Brosseau. "Software design patterns for robotics: Solving integration problems with MARIE." In Workshop of Robotic Software Environment, IEEE International Conference on Robotics and Automation. 2005.

[13] Jawawi, Dayang NA, Safaai Deris, and Rosbi Mamat. "An approach to reusable software for mobile robot applications through analysis patterns." (2005): ICMT-144.

[14] Batinica, Aleksandar, Srđan Savić, Milutin Nikolić, Mirko Raković, and Branislav Borovac. "Dual BLDC motor driver for arm of humanoid robot MARKO.", In Proceedings of the 3rd international conference and workshop Mechatronics in Practice and, Education – MECHEDU, 2015

[15] V. Potkonjak, M. Vukobratović, K. Babković, and B. Borovac, "General model of dynamics of human and humanoid motion: feasibility, potentials and verification," *International Journal of Humanoid Robotics*, vol. 3, no. 01, pp. 21–47, 2006.

[16] Asfour, Tamim, and Rüdiger Dillmann. "Human-like motion of a humanoid robot arm based on a closed-form solution of the inverse kinematics problem." In Intelligent Robots and Systems, 2003.(IROS 2003). Proceedings. 2003 IEEE/RSJ International Conference on, vol. 2, pp. 1407-1412. IEEE, 2003.

[17] Tevatia, Gaurav, and Stefan Schaal. "Inverse kinematics for humanoid robots." In Robotics and Automation, 2000. Proceedings. ICRA'00. IEEE International Conference on, vol. 1, pp. 294-299. IEEE, 2000.

[18] Hildenbrand, Dietmar, Julio Zamora, and Eduardo Bayro-Corrochano. "Inverse kinematics computation in computer graphics and robotics using conformal geometric algebra." Advances in applied Clifford algebras 18, no. 3-4 (2008): 699-713.

[19] Kim, Hyunchul, Levi Makaio Miller, Nancy Byl, Gary M. Abrams, and Jacob Rosen. "Redundancy resolution of the human arm and an upper limb exoskeleton." Biomedical Engineering, IEEE Transactions on 59, no. 6 (2012): 1770-1779.

[20] Raković, Mirko, Branislav Borovac, Milutin Nikolić, and Srđan Savić. "Realization of Biped Walking in Unstructured Environment Using Motion Primitives." Robotics, IEEE Transactions on 30, no. 6 (2014): 1318-1332.

[21] Nikolić M., Borovac B., Raković M., Savić S. "A further generalization of task-oriented control through tasks prioritization." International Journal of Humanoid Robotics 10, no. 03 (2013): 1350012.

[22] Savić, Srđan, Mirko Raković, and Branislav Borovac. "Mechanical design and control algorithm for the arm of humanoid robot MARKO." In 6th PSU-UNS International Conference on Engineering and Technology ICET 2013. 2013.